Finance OS / APAR

Accounts Payable & Receivable — autonomously managed by AI agents, human-supervised by exception

Product Overview & User Guide

Version 2.0 · research build · June 2026

Built & authored by **Mohan Chandolu**

An AI/agentic finance back office: software agents ingest invoices and payments, perform 3-way matching, apply incoming cash, drive collections, and flag anomalies — while a human analyst supervises by exception rather than doing manual data entry.

1 · Overview

The problem

Accounts Payable and Receivable are high-volume, rules-heavy, and audited. A mid-size company processes tens of thousands of invoices a month — today largely by hand: keying data, reconciling spreadsheets, three-way-matching, chasing overdue accounts. It is slow, error-prone, and where money quietly leaks through duplicate payments and missed terms.

What Finance OS does

Finance OS turns that back office into a supervised, agentic system. Each workflow has a dedicated agent that does the routine work and escalates only what needs judgment. The analyst — our persona, Dana Okafor — reviews exceptions and tunes the agents instead of processing every line.

Principles

- **Supervise by exception.** Agents auto-handle the clean majority; humans see only what's ambiguous or high-value.
- **Deterministic where it matters, AI where it helps.** Rules do the math (matching, reconciliation); a language model only explains the gray-area cases — it never approves a payment.
- **Every action gated & logged.** Auto-actions fire only above an agent's confidence threshold; everything lands in an immutable audit trail with its reason and source documents.
- **Your data stays yours.** The reasoning model can run locally (Ollama / vLLM); connectors are vendor-swappable; the system runs offline on a mock seam.

Who it's for

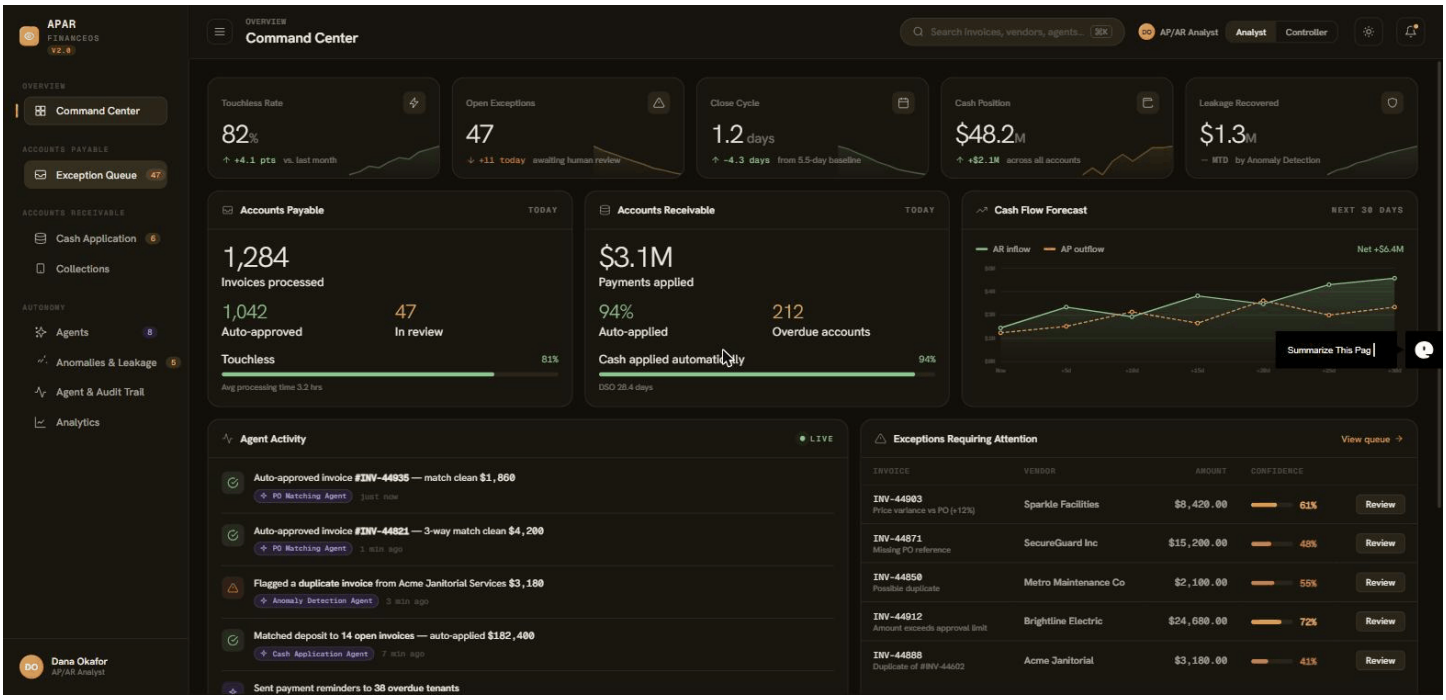
AP/AR analysts and controllers in finance teams; the platform is generic and resellable to any organization once hosted.

2 · Guided walkthrough

The product is seven screens grouped as Overview, Accounts Payable, Accounts Receivable, and Autonomy. Each runs on live, persisted, agent-computed data.

2.1 Command Center

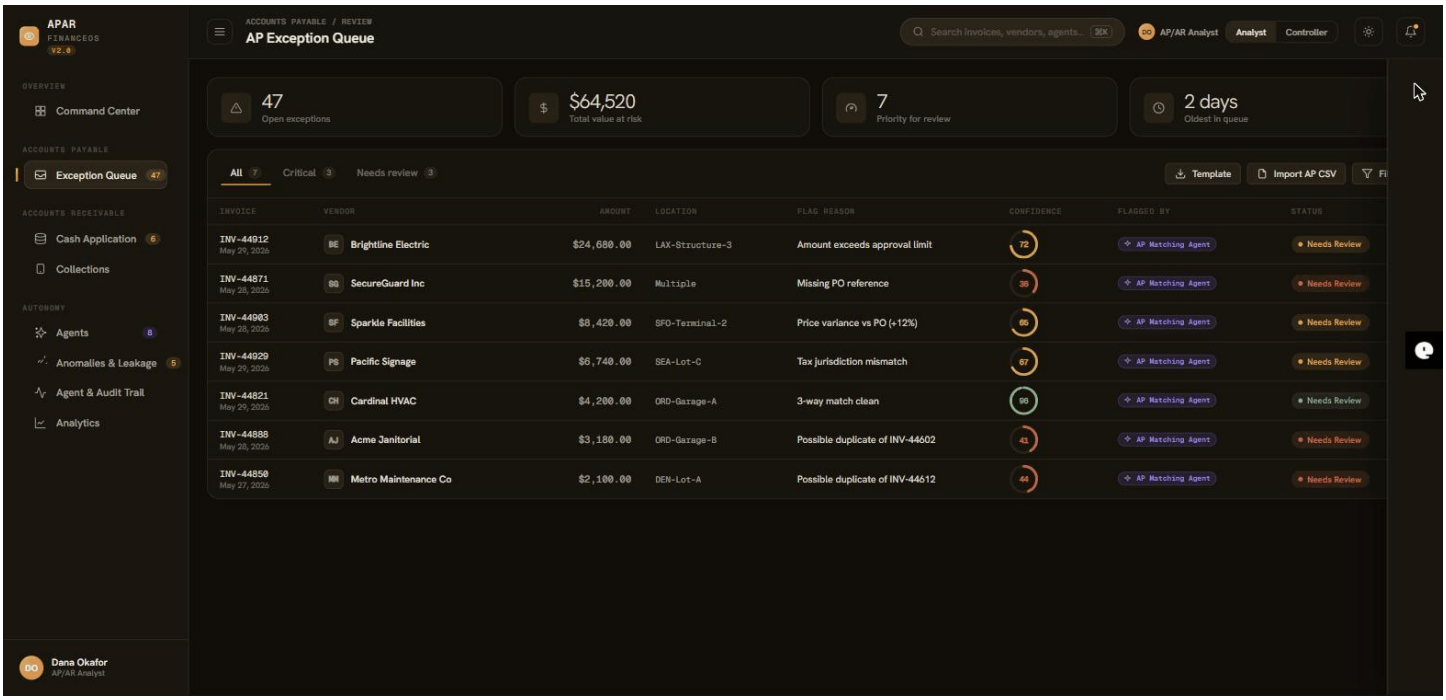
The Command Center opens with fleet-level metrics and an **Agents at work** strip showing every agent live, its mode and threshold, and today's touchless rate. AP and AR health tiles and a 30-day cash-flow forecast give an at-a-glance read of the back office.



The home view: outcome KPIs a CFO cares about (touchless rate, open exceptions, close cycle, cash position, leakage recovered).

2.2 Exception Queue (AP)

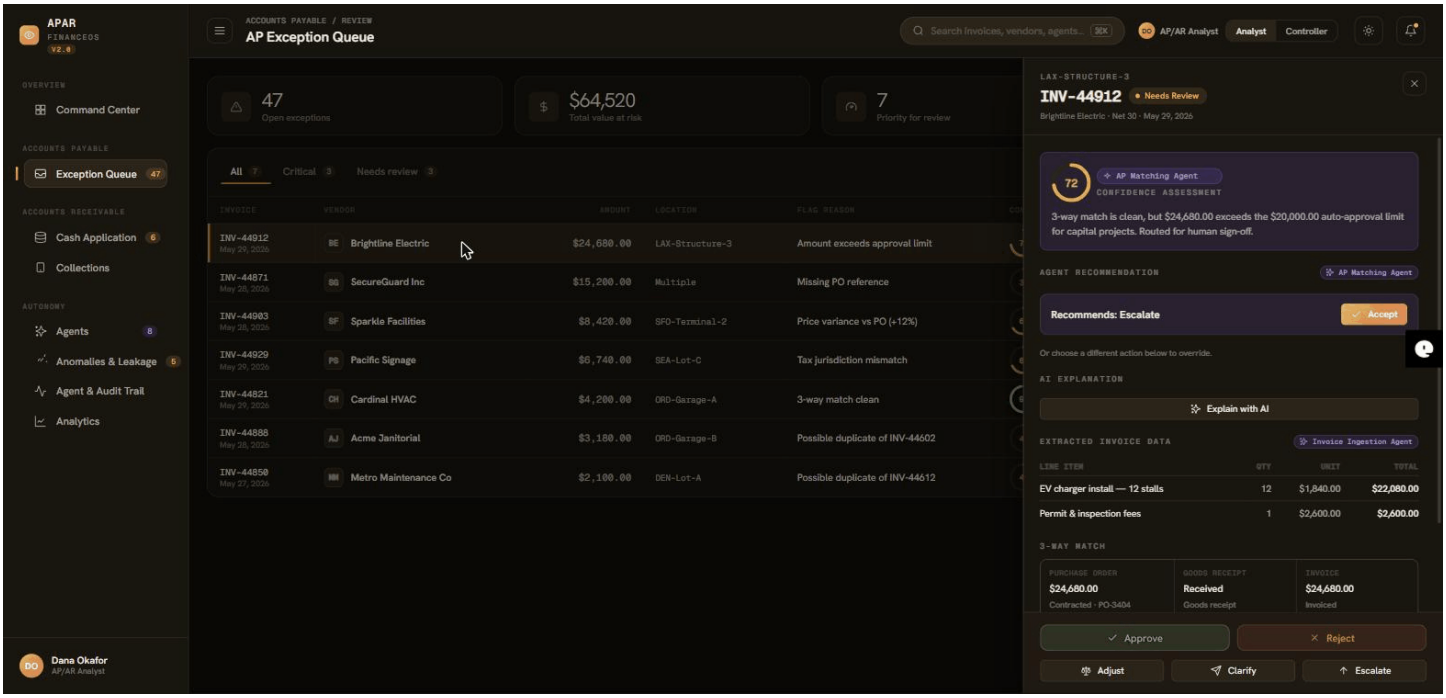
The agent ingests invoices, runs a deterministic three-way match (PO ↔ goods receipt ↔ invoice), and auto-clears the clean ones. The queue holds the exceptions — price variances, missing POs, duplicates, over-limit and tax-jurisdiction cases — each with a flag reason, amount at risk, confidence, and the agent that raised it.



What the AP Matching agent couldn't safely auto-approve — the only invoices a human needs to touch.

2.3 Reviewing an exception

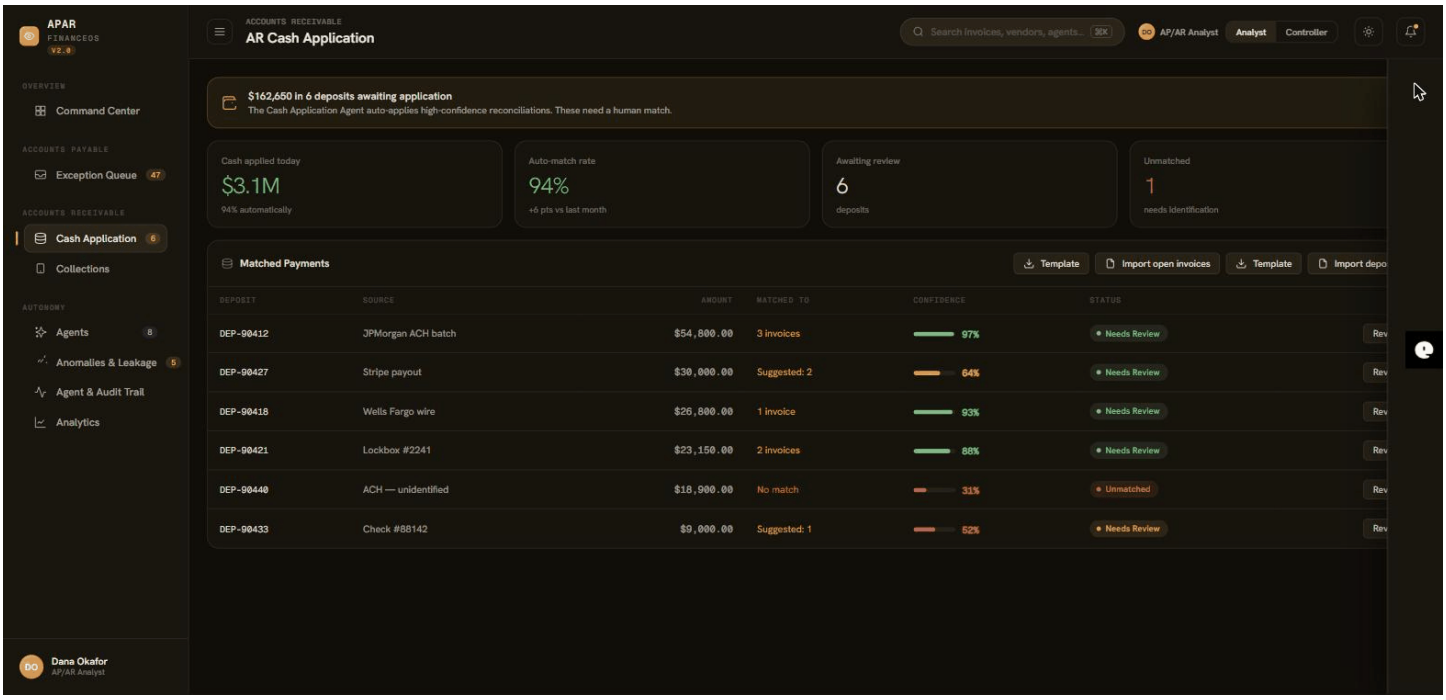
Opening an exception shows the PO/GR/invoice side-by-side with the mismatch highlighted, the line items, a confidence ring, and the contract/policy it's grounded in. The agent gives a recommendation (Accept in one click, or Override). **Explain with AI** drafts a controls-analyst narrative and a suggested disposition; the model badge shows which model produced it (local Ollama or hosted), and only the discrepancy — never the vendor master — is sent to it.



The detail drawer: 3-way match evidence, the agent's recommendation, and an AI-drafted explanation.

2.4 Cash Application (AR)

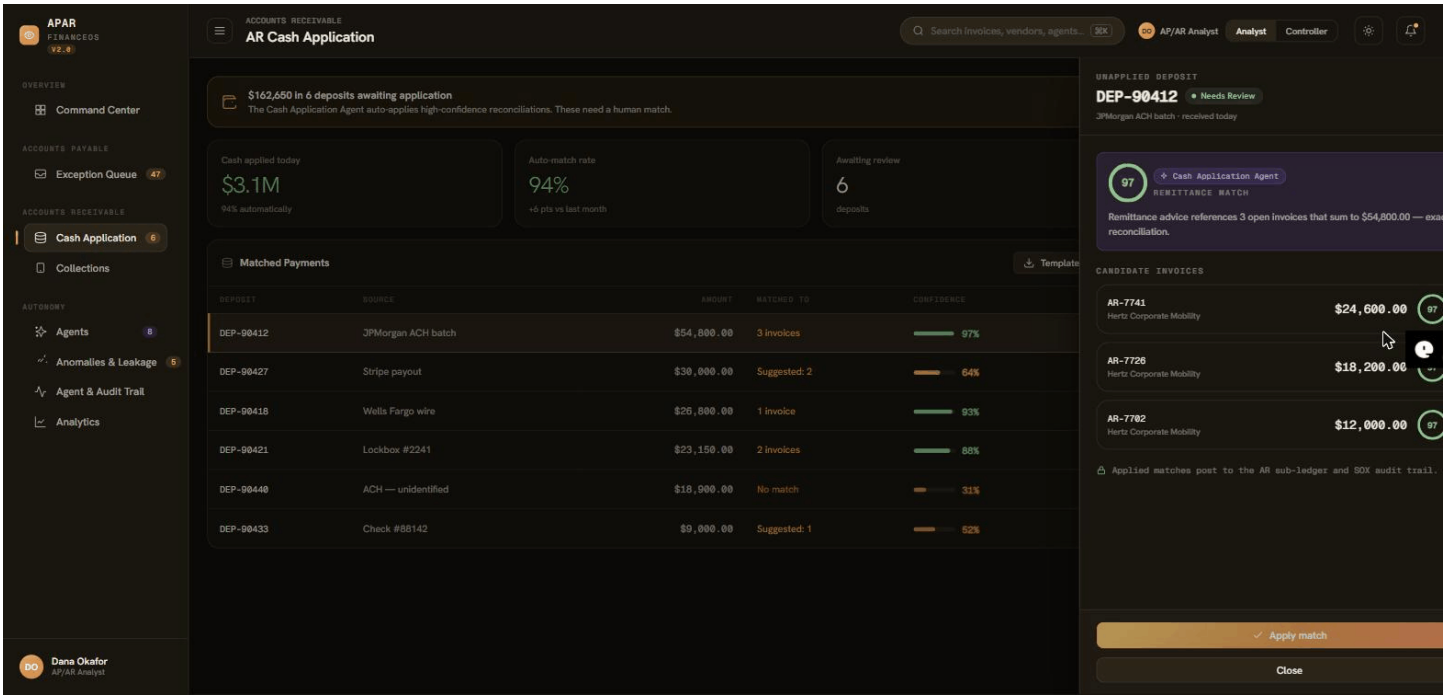
The Cash Application agent reconciles deposits against open AR invoices. High-confidence matches auto-apply; the rest — partial matches, unidentified ACH — come here for a human, pre-filled with the agent's proposed invoice set and confidence.



Incoming deposits matched to open invoices — exact, remittance-hint, then fuzzy.

2.5 Applying a deposit

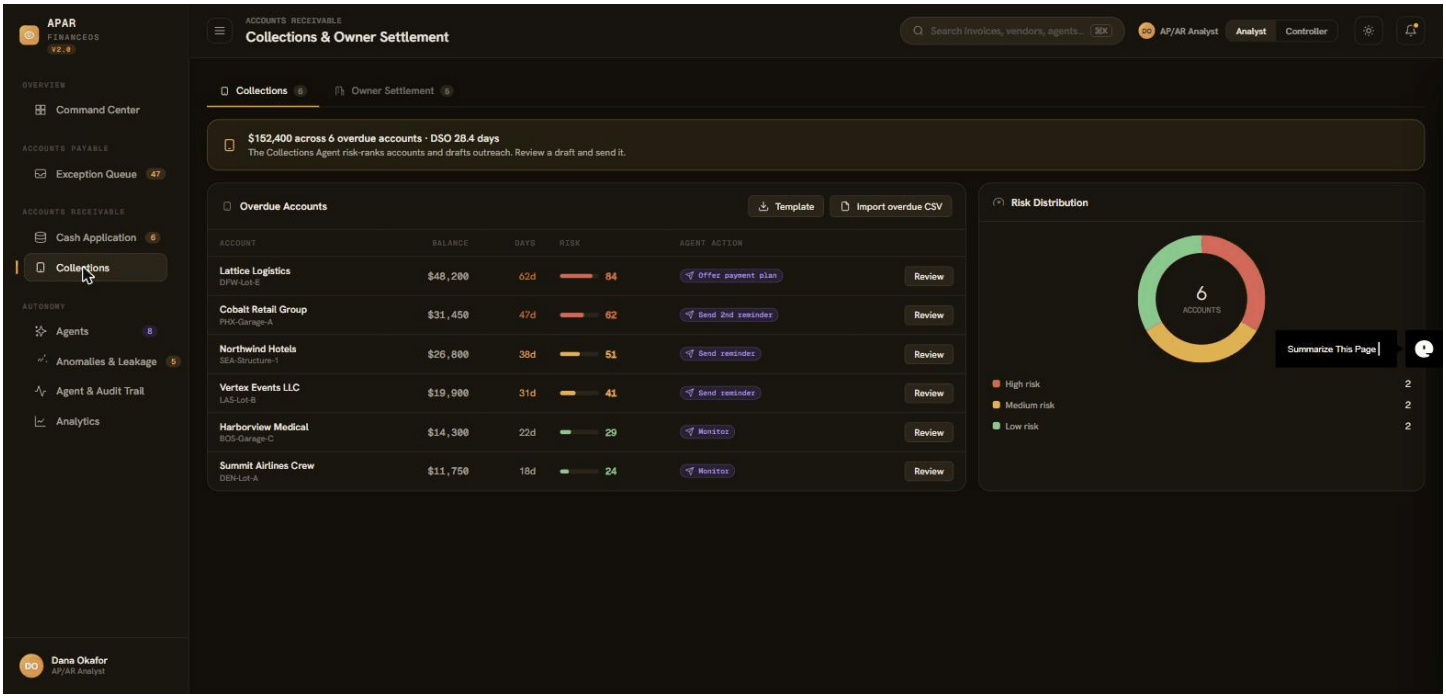
Opening a deposit shows the remittance match — the open invoices the agent believes it pays, each with its own confidence — summing to the deposit amount. Apply posts the reconciliation to the AR sub-ledger and the audit trail; the human is the check on the agent, not the data-entry clerk.



The AR detail drawer: the agent's proposed invoice set for a deposit, ready for one-click apply.

2.6 Collections

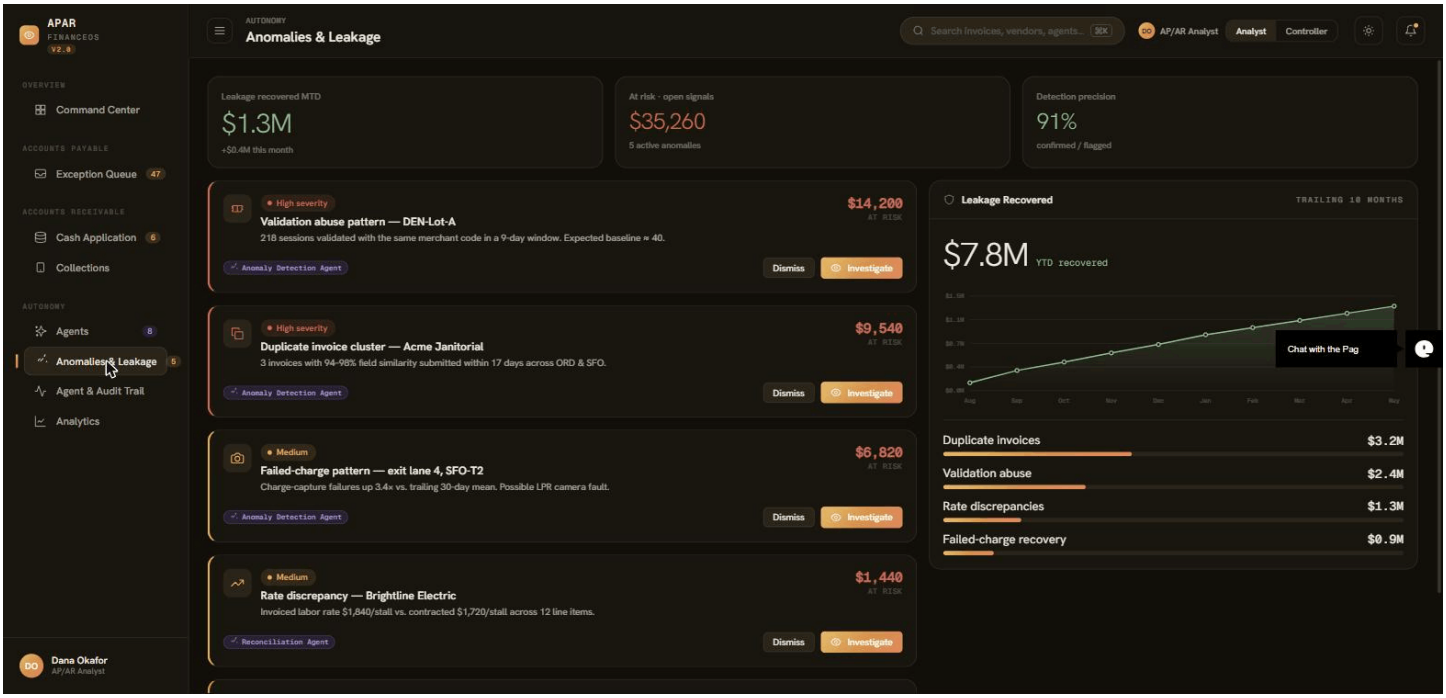
The Collections agent scores each overdue account by days-past-due and balance, prioritizes them, and drafts the outreach message — but a person sends anything customer-facing. The risk distribution shows the book at a glance.



Risk-ranked overdue accounts with agent-drafted outreach (a human sends).

2.7 Anomalies & Leakage

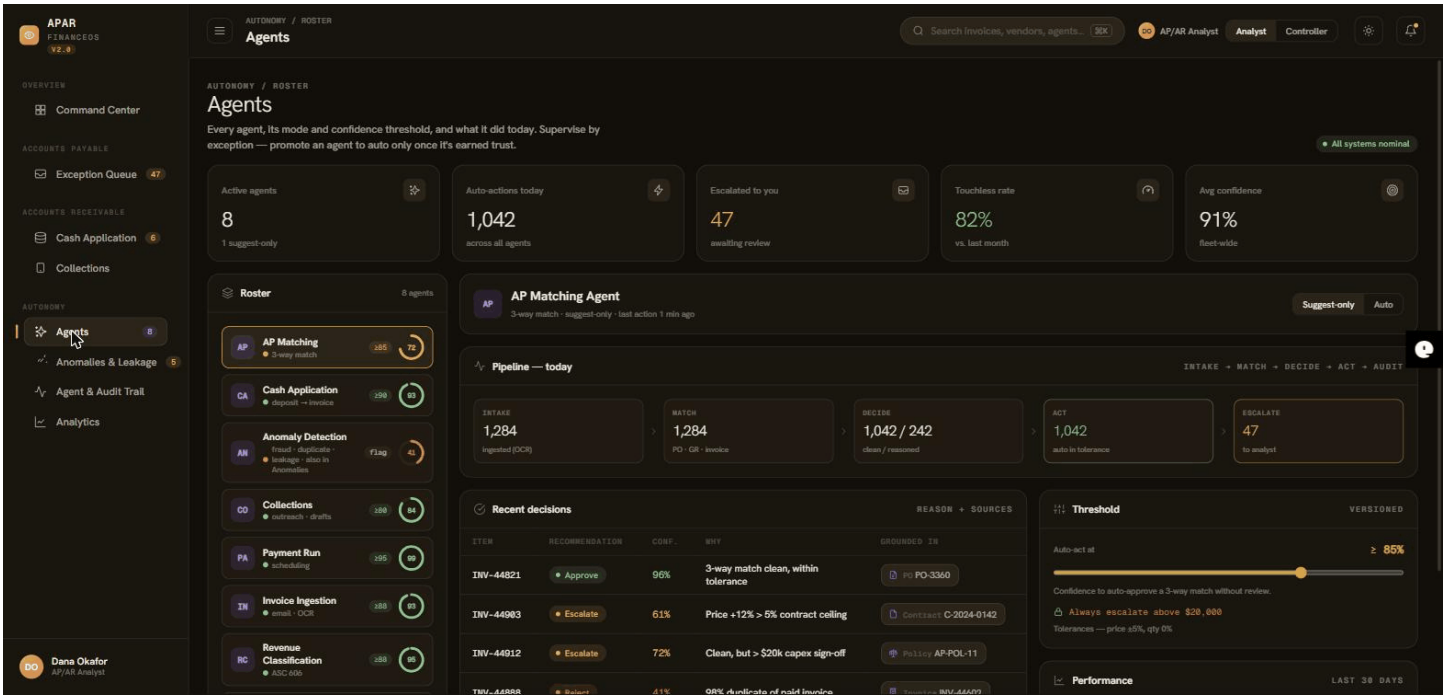
The Anomaly agent watches the ledger and surfaces severity-ranked signals with the dollars at risk, plus a leakage-recovered trend — the ROI story for the program.



Where the money comes back: duplicate clusters, validation abuse, charge-capture faults, off-contract spend.

2.8 Agents console — autonomy

Each agent shows its pipeline (intake → match → decide → act → escalate), recent decisions with reasons and sources, and an editable confidence threshold. Agents start in **suggest-only** mode and are dialed toward auto-action; a global kill switch forces everything back to suggest-only in an incident. Changing autonomy requires the Controller role (segregation of duties).



Every agent in one place; promote each from suggest-only to auto as it earns trust.

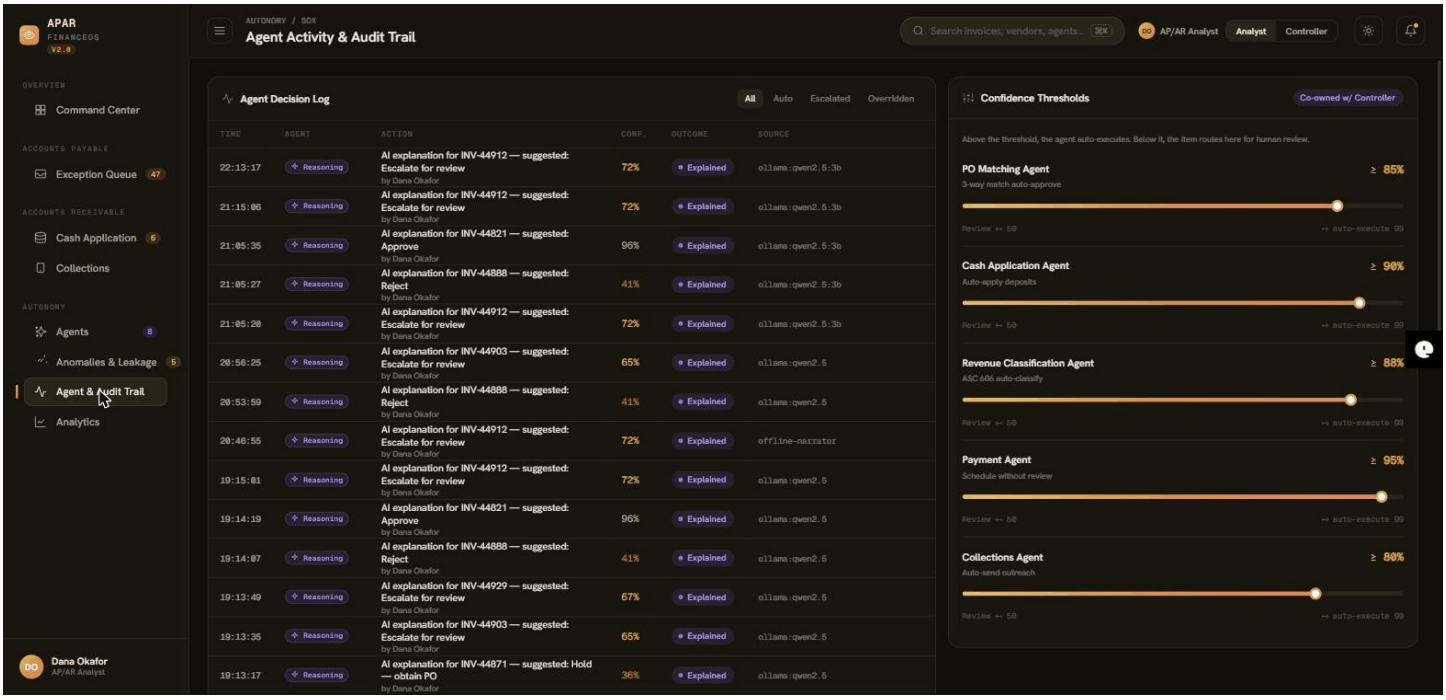
2.9 Analytics

Touchless rate, cost per invoice, close-cycle time, DSO, and human-override rate over time — alongside a live panel that reflects this session's real agent activity. Promote an agent to auto and you watch the touchless rate climb and the open queue fall.

Historical trends plus a live-this-session block computed from the audit trail.

2.10 Agent & Audit Trail

Every auto-action, human resolution, threshold change, and AI explanation — with the model that produced it — is appended here, never edited. Confidence thresholds are co-owned with the Controller. When an auditor asks 'why was this paid,' the answer is one click with the evidence attached.



The SOX story: an immutable, queryable log of every action and who took it.

2.11 Importing your data (CSV / ERP)

Finance OS reads real data, not just the bundled sample. Download a CSV template from the Exception Queue or Cash Application screen, fill in your invoices, POs, receipts, open invoices, or deposits, and import — the agents re-run on your numbers in seconds (duplicate invoices are auto-detected by vendor and amount). The same connector interface that reads a CSV today plugs into NetSuite, SAP, or QuickBooks tomorrow; bank feeds and OCR intake sit behind the same seam, and nothing is hard-wired to a vendor.

Appendix A · Technical architecture

Four layers, front to back, with a shared schema and an event/audit log cross-cutting all of them.

apps/web	React 19 + Vite + Zustand — the seven-screen UI (Espresso theme), typed against the shared schema
services/api	FastAPI BFF — read models + command endpoints, auth/RBAC, audit write-through, CSV ingestion
services/agents	Agent layer — deterministic engines (3-way match, cash match, risk scoring) + LLM reasoning, under confidence-threshold gates
packages/shared	Domain schema (types) shared by web and API
packages/connectors	Abstract ERP / bank / ingestion contracts + adapters (mock, file/CSV); vendor SDKs slot in here
persistence	stdlib sqlite3 — exceptions, deposits, collections, append-only audit, agent config (Postgres-ready behind repo.py)

The agent pipeline

Intake → Enrich & Match → Decide → (Auto-act | Escalate) → Audit. Clean cases are decided by rules; ambiguous ones get an LLM-drafted rationale. Confidence is compared to the agent's threshold: above it (and only when not in suggest-only mode) the agent auto-acts via a connector and logs; below it the item escalates to the analyst, pre-filled with the recommendation.

Security & control

- **RBAC** — Analyst (least-privilege) and Controller; tuning autonomy and resolving items ≥ $20k require Controller (segregation of duties).
- **Immutable audit** — append-only; covers agent and human actions, including which model produced each AI explanation.
- **Secrets** via environment / secret manager, never in the repo; parameterized SQL; CORS allow-list; global kill switch.
- **Data minimization** — the LLM sees only the discrepancy; it can run fully local (Ollama / vLLM) so data never leaves your infrastructure.

The reasoning model — why local Ollama + Qwen2.5

Deterministic rules own every disposition; the language model is used only to **explain** gray-area exceptions in a controls-analyst voice and suggest a disposition — it never approves a payment. Because the input is financial data, the model runs **locally** rather than calling an outside API.

- **Ollama (local / self-hosted).** Ollama runs open-weight models on your own hardware behind an OpenAI-compatible endpoint (it also works against vLLM/TGI). Data never leaves your infrastructure — the key requirement for AP/AR documents — and there's no per-token cost or external dependency.
- **Qwen2.5:3b is the default.** Among locally-runnable models it hit the best latency/quality balance on CPU: clean three-way-match and duplicate-invoice narratives at roughly 7–8 seconds per explanation. The larger Qwen2.5 (7B) is higher quality but about 3× slower, and the 0.5B variant is fast but unreliable — so the 3B is the sweet spot for a responsive reviewer drawer.
- **Repeatable for audit.** The model runs at temperature 0 so the same exception yields the same explanation, and the exact model string (e.g. *ollama:qwen2.5:3b*) is written to the audit trail with every explanation.
- **Data-minimized.** Only the structured discrepancy is sent to the model — never the vendor or customer master.
- **Swappable & safe to fail.** A single setting (LLM_PROVIDER) switches between local Ollama, a hosted provider (Anthropic), or a dependency-free offline narrator; if the chosen model is unreachable the system falls back to the offline narrator and never errors.

Technology glossary

Plain-English notes on the tools named above, for non-engineering readers.

React	A JavaScript library for building user interfaces out of reusable components; it keeps what's on screen in sync with the underlying data.
TypeScript	JavaScript with type checking added — it catches mistakes (e.g. a wrong field name) before the app runs, which matters in a finance UI.
Vite	The build tool/dev server for the front-end: it bundles the React/TypeScript code into the fast static site the browser loads.
Zustand	A small state-management library for React — the single place the UI keeps shared data (the current queue, the logged-in role) so screens stay consistent.
Python	The programming language the back-end and the agents are written in; widely used for data and AI work.
FastAPI	A Python framework for building web APIs — the layer the browser calls to read data and send commands (approve, apply, send).
BFF (Backend-for-Frontend)	An API shaped specifically for this UI: it returns exactly the data each screen needs rather than raw database tables.
SQLite	A lightweight, file-based database (no separate server) used here to store exceptions, deposits, and the audit log; swappable for Postgres later.
Postgres	A full client–server relational database for production scale — the upgrade path from SQLite when volume grows.

Ollama / vLLM	Runtimes that let a large language model run on your own hardware, so the AI explanations work without sending data to an outside service. The default model here is Qwen2.5:3b.
LLM	Large Language Model — the AI that drafts the plain-English explanation of a gray-area exception (it explains, it never approves a payment).
RBAC	Role-Based Access Control — permissions tied to a role (Analyst vs Controller) so only authorized people can change thresholds or approve large items.
CORS	A browser security rule controlling which web origins may call the API; locked to an allow-list here.
ERP	Enterprise Resource Planning system (e.g. NetSuite, SAP, QuickBooks) — the system of record for invoices and payments the connectors read from.
3-way match	The AP control that checks a purchase order, the goods receipt, and the invoice agree before paying — the core rule the AP agent automates.

Appendix B · What's built so far

Delivered (Phases 0–4 of the design spec, plus LLM reasoning and ingestion):

- **Foundation** — monorepo, shared schema, FastAPI, connector contracts + mock seam; all seven screens wired to the API.
- **AP exceptions** — deterministic tolerance-aware 3-way match engine; AP Matching agent (suggest-only); persistence + immutable audit.
- **AR cash application** — deposit→invoice match engine (exact / remittance / batch / fuzzy); apply flow.
- **Collections, Anomalies, Analytics** — risk-scored outreach drafts; severity-ranked leakage detection; analytics with a live-session block from the audit trail.
- **Autonomy** — promote an agent suggest-only → auto and it auto-acts on threshold-clearing items, moving the touchless rate.
- **Hardening** — Auth + RBAC with segregation of duties; security review (SECURITY.md).
- **Data in** — File/CSV ERP adapter + AP/AR/Collections CSV upload with templates and auto duplicate detection.
- **LLM-assisted reasoning** — swappable provider (local Ollama / hosted / offline narrator), data-minimized, audit-logged with the model used.
- **Quality** — 16 unit tests across the match engines; clean web build.

Next product priorities (PM view)

The top three features that would move the product furthest from here — each extends autonomy into a part of the workflow the agents don’t yet own end-to-end.

- **1 · Autonomous payment execution & run orchestration.** Today the agents approve invoices but stop short of paying them. Close the AP loop: schedule and execute payment runs over real rails (ACH / wire / virtual card) with dual-approval, positive-pay, and automatic early-payment-discount capture. This turns ‘approved’ into ‘paid’ untouched and converts the touchless rate into hard cash savings — the single biggest unlock.

- **2 · Two-way supplier & customer portal.** A branded portal where suppliers submit invoices and check payment status, and where the Collections agent's outreach becomes a conversation — customers dispute, promise-to-pay, or self-serve a payment plan. Cuts inbound 'where's my payment?' volume and shortens DSO by removing the email back-and-forth; it also feeds the agents structured intake instead of PDFs.
- **3 · Closed-loop learning from analyst decisions.** Every human override is training signal. Use it to auto-tune each agent's confidence thresholds and refine the match / risk models, so the touchless rate climbs on its own and the same exception is rarely escalated twice. This is the compounding moat of an agentic system — the product gets more autonomous the longer it runs.
- **4 · Supervisor / agent control plane (V3).** A deterministic control plane that watches the agents themselves — tracking each one's override rate, confidence drift, volume, and error rate against a baseline. It acts as a circuit breaker: an agent that degrades or behaves anomalously is auto-demoted to suggest-only and a human is paged, and any action outside an agent's allowed envelope is gated. Crucially the gating logic is rule-based, not another autonomous LLM — the guardrail can't itself go rogue. This is the governance counterpart to closed-loop learning; full design in *V3_TECH_SPEC.md*.

Platform & infrastructure (deferred — needs real credentials / infra)

- Real vendor-SDK ERP / bank / OCR adapters (the swap seam is a single file).
- Real identity provider + JWT, TLS / rate limiting, Postgres, async intake for volume, and deployment.

Appendix C · Project files & layout

What every file and folder in the repository is for. The project is an npm + Python monorepo: a React front-end, a FastAPI back-end, the agent layer, and shared packages, with sample data and docs around them.

Root — docs & config

CLAUDE.md	The standing 'how to work here' brief: what the project is, the stack, conventions, guardrails, and the phase-by-phase status. Read first in any session.
V2_DESIGN_SPEC.md	The V2 architecture and roadmap — the target layout, the agent/threshold model, and the phased plan this build follows.
V1_HANDOFF.md	Snapshot of where the V1 prototype left off (the front-end-only mock app) — the V1→V2 handoff.
V2_HANDOFF.md	Snapshot of where V2 landed and the context for V3 — the V2→V3 handoff.
V3_TECH_SPEC.md	Proposed V3: the Supervisor / agent control plane — goals, what it monitors, circuit-breaker design, data model, and phased plan.
README.md	How to install and run the project (front-end and back-end), for a new developer.
SECURITY.md	The security review — what's hardened (RBAC, immutable audit, parameterized SQL, secrets, CORS) vs. deferred, plus the connector vendor-swap guide.
CHANGELOG.md	Human-readable log of what changed across the phases.
package.json / package-lock.json	npm workspace manifest (root) tying the front-end and shared packages together, plus the locked dependency versions.

.gitignore	Files Git ignores — build output, node_modules, the local SQLite DB, virtualenvs, secrets.
APAR FinanceOS.html	A standalone single-file export of the UI (a self-contained preview artifact).

apps/web — the front-end (React + Vite + TypeScript)

src/screens/	The seven screens: Command Center, Exception Queue, Cash Application, Collections, Anomalies, Agents, Audit, Analytics.
src/chrome/	App shell — the sidebar (with the V2.0 tag) and the top bar (search, role switcher).
src/components/	Reusable UI pieces — charts, status pills, confidence bars, the CSV-import control.
src/store/	Zustand store: shared client state (current role/user, queues, async actions like resolve).
src/api/	The client that calls the back-end (auth token, fetch/command functions, CSV upload).
src/data/	The bundled mock seed data + re-exported types — lets the UI run offline if the API is down.
src/styles/	The Espresso theme (CSS variables, component classes).
index.html, vite.config.ts, tsconfig*.json, eslint.config.js	Entry HTML, Vite build config (pinned dev port), TypeScript settings, and lint rules.
.env.example	Sample front-end environment (e.g. the API base URL); copy to .env locally.
dist/	The built static site Vite produces — what the API serves at /app.

services/ — the back-end & agents (Python)

api/app/main.py	FastAPI app entry — wires the routers, serves the built UI, seeds the DB on startup.
api/app/routers/	One file per endpoint group: exceptions, deposits, collections, agents, anomalies, analytics, auth, ingest, health.
api/app/db.py, repo.py, store.py	SQLite schema, the single persistence layer (repo.py — the Postgres-swap point), and an in-memory helper.
api/app/auth.py, config.py, models.py	RBAC roles/tokens, settings (e.g. the $20k segregation-of-duties limit), and the API data shapes (Pydantic).
api/requirements.txt, pyproject.toml	Back-end Python dependencies and project metadata.
api/financeos.db	The local SQLite database file (created/seeded at runtime; gitignored).
agents/financeos_agents/matching.py, cash_matching.py	The deterministic engines: tolerance-aware 3-way match (AP) and deposit→invoice match (AR).
agents/.../ap_matching.py, cash_application.py, collections.py, anomaly.py	The agents that run those engines under a confidence-threshold gate and produce the queues.
agents/.../llm.py, reasoning.py	The swappable LLM provider (local Ollama / hosted / offline narrator) and the explain logic.

agents/.../thresholds.py, pipeline.py	The threshold gate (suggest-only vs auto) and the intake→decide→act→audit pipeline scaffold.
agents/tests/	Unit tests for the match engines (run with python -m unittest).

packages/ — shared code

shared/src/types.ts	The domain schema (Exception, Deposit, Collection, Anomaly, Audit, Threshold …) shared by the web app and the API — the single source of truth for data shapes.
connectors/financeos_connectors/	Abstract ERP / bank / ingestion contracts plus adapters: a mock seam for offline demos and a file/CSV adapter; real vendor SDKs slot in behind the same interfaces.

infra/, demo/, design/

infra/.env.example	Back-end environment/secrets template (DB path, ERP mode, LLM provider, SoD limit) — secrets live here, never in code.
infra/data/*.csv	Sample AP invoices, AR open invoices, deposits, and overdue accounts — used in file/CSV mode and as import templates.
demo/	This user manual and its build script (build_manual.py), the screenshots (screens/), and the VC demo script.
design/agents-ux-wireframes.html	The Espresso wireframes explored for the Agents screen before it was built.

Finance OS / APAR — V2 research build. Generated from the live product.